Research Assistant: An Integrated NLP and Generative AI Platform for Academic Research Support

Daxa Antiya
Department of Computer Science & Engineering
Indus University
Ahmedabad, Gujarat, India

Viren Shah
Department of Computer Science & Engineering
Indus University
Ahmedabad, Gujarat, India

Rishabh Chopda
Department of Computer Science & Engineering
Indus University
Ahmedabad, Gujarat, India

***Abstract*—The rapid growth of academic publications has made research-paper discovery, analysis, synthesis, and academic writing increasingly time-consuming for students and researchers. Conventional workflows often require users to switch between multiple independent tools for finding relevant papers, extracting information from research documents, generating summaries, preparing literature reviews, evaluating textual similarity, paraphrasing content, answering research questions, and generating citations. This paper presents Research Assistant, an integrated web-based platform that combines Natural Language Processing (NLP), Machine Learning, information retrieval, document processing, and Generative Artificial Intelligence (AI) to support these activities within a unified workflow. The system retrieves research papers from the arXiv API and applies the pretrained `all-MiniLM-L6-v2` Sentence Transformer model for semantic paper retrieval and similarity analysis. Uploaded PDF documents are processed using PyMuPDF for text extraction. Generative AI capabilities are provided through the Groq API using the `openai/gpt-oss-120b` model for summarization, literature-review generation, conversational research assistance, and analytical interpretation. The Humanizer module uses the pretrained `tuner007/pegasus_paraphrase` model for sentence-level paraphrasing and `all-MiniLM-L6-v2` for semantic candidate evaluation, supported by protected-information validation to reduce factual and technical-term corruption. A hybrid plagiarism-oriented analysis combines n-gram overlap with semantic similarity. Functional testing across the implemented modules demonstrated successful operation without genuine application failures in the executed test suite. The proposed platform demonstrates how multiple NLP and Generative AI techniques can be integrated into a single academic research workflow.**

***Index Terms*—Natural Language Processing, Machine Learning, Semantic Search, Sentence Transformers, Generative AI, Research Paper Analysis, Text Paraphrasing, Plagiarism Analysis, Academic Research Assistant.**

I. Introduction

The continuous growth of scientific and technical literature has significantly increased the amount of information that students and researchers must examine during the research process. Finding relevant publications, understanding lengthy research papers, comparing previous studies, preparing literature reviews, evaluating textual similarity, and formatting citations are often repetitive and time-consuming activities. Conventional academic search systems may primarily rely on keyword matching, which can fail to capture the semantic relationship between a research query and the content of a paper.

Recent developments in Natural Language Processing (NLP) and Machine Learning have enabled more effective approaches for representing and comparing textual information. Sentence embedding models can convert sentences and documents into numerical vectors that capture semantic information. Sentence-BERT, for example, introduced an efficient approach for producing semantically meaningful sentence embeddings that can be compared using cosine similarity, making such models suitable for semantic retrieval and similarity applications [1]. Academic repositories such as arXiv also provide programmatic access to research-paper metadata and abstracts, enabling integration with software-based research workflows [4].

Large Language Models (LLMs) have further expanded the capabilities of academic-assistance systems. Generative AI can summarize documents, synthesize information, answer research-related questions, transform text, and assist with academic writing. However, using a language model alone does not provide a complete research workflow. An effective research assistant benefits from combining information retrieval, semantic representation, document processing, textual similarity, and generative capabilities.

To address these requirements, this work presents Research Assistant, a web-based academic research support system integrating Machine Learning, NLP, information retrieval, PDF processing, and Generative AI. The system retrieves research papers from arXiv and ranks them according to semantic relevance using the pretrained `all-MiniLM-L6-v2` Sentence Transformer model. It also processes uploaded PDF docu-

ments, generates summaries, supports literature-review generation, performs plagiarism-oriented textual similarity analysis, provides ML-based text humanization, answers research-related questions, and generates citations in multiple academic formats.

The Machine Learning component primarily uses semantic embeddings. A user's research query and the title-abstract representation of retrieved papers are converted into embeddings, after which cosine similarity is used to estimate semantic relevance. The same embedding model is used for sentence-level comparison during plagiarism analysis and for semantic evaluation of paraphrase candidates in the Humanizer. Generative AI components use the `openai/gpt-oss-120b` model through the Groq API for generation-oriented tasks.

A major contribution of the project is the integration of multiple research-support functions into a single application rather than treating paper retrieval, document analysis, writing assistance, similarity analysis, and citation generation as independent tools. The resulting system provides an end-to-end workflow beginning with research-paper discovery and extending to document analysis, synthesis, textual evaluation, paraphrasing, and citation generation.

The remainder of this paper is organized as follows. Section II discusses related work. Section III presents the problem statement and objectives. Section IV describes the proposed system and architecture. Section V explains the methodologies used by the major modules. Section VI discusses implementation details. Section VII presents the experimental results and discussion. Section VIII discusses limitations and future scope. Section IX concludes the paper.

II. Related Work

Semantic text representation has become an important component of modern information-retrieval and NLP systems. Reimers and Gurevych introduced Sentence-BERT (SBERT), which uses a Siamese or triplet network structure to generate sentence embeddings that can be efficiently compared using cosine similarity [1]. This approach provides a practical foundation for semantic search because a query and a collection of documents can be independently encoded and compared.

BERT introduced bidirectional Transformer-based contextual representations and established a foundation for many downstream NLP applications [2]. Sentence-level embedding models derived from Transformer architectures provide an efficient way of comparing textual content while avoiding repeated full pairwise processing.

Academic information retrieval can also be supported through programmatic access to scholarly repositories. The arXiv API provides a structured interface for searching academic papers and retrieving metadata such as titles, authors, abstracts, publication information, and links [4]. Such interfaces allow scholarly retrieval to be incorporated directly into research-support applications.

Large Language Models have expanded automated academic assistance through capabilities such as summarization, question answering, information synthesis, and text transformation. However, generated responses may depend strongly on prompts and supplied context. Consequently, integration with retrieval and document-processing components is important when designing systems for research-oriented use.

Plagiarism and textual-overlap analysis can benefit from combining lexical and semantic approaches. Exact or near-exact phrase matching can identify reused expressions, while semantic similarity can identify conceptually related sentences even when their wording differs. A hybrid approach therefore provides broader evidence than either lexical or semantic comparison alone.

Paraphrase generation is another important NLP task. Sequence-to-sequence Transformer models can generate alternative formulations of existing text while attempting to preserve its underlying meaning. In the proposed system, a pretrained Pegasus paraphrasing model is combined with semantic similarity scoring and protected-information validation to make paraphrasing more suitable for technical and academic text.

Retrieval-Augmented Generation (RAG) has also been proposed for knowledge-intensive NLP tasks by combining retrieval with language-model generation [3]. Although the current system does not implement a complete RAG architecture, its combination of academic retrieval and Generative AI provides a foundation for future retrieval-grounded research assistance.

Despite the availability of individual tools for these tasks, there remains a practical need for integrated systems that combine academic retrieval, semantic analysis, document processing, generative assistance, paraphrasing, textual similarity analysis, and citation generation. Research Assistant addresses this need through a unified application.

III. Problem Statement and Objectives

A. Problem Statement

Students and researchers frequently depend on separate tools for finding academic papers, reading and summarizing documents, preparing literature reviews, checking textual similarity, rewriting text, answering research questions, and formatting citations. Switching between multiple systems increases workflow complexity and may require repeated manual processing of the same information.

The problem addressed by this project is therefore the development of an integrated research assistant capable of supporting multiple stages of academic research through semantic retrieval, NLP-based analysis, PDF processing, Generative AI, and automated text transformation.

B. Objectives

The primary objectives of Research Assistant are:

- To provide semantic research-paper retrieval using a pretrained Sentence Transformer model.
- To retrieve academic papers programmatically using the arXiv API.

- To extract and analyze textual content from uploaded PDF research papers.
- To generate summaries from research documents and textual inputs.
- To support automated literature-review generation from retrieved research papers.
- To identify possible textual overlap using n-gram and semantic similarity methods.
- To provide meaning-preserving text paraphrasing through a pretrained Transformer model.
- To preserve important numerical and technical information during paraphrasing.
- To provide an interactive research-oriented conversational assistant.
- To generate citations in multiple academic citation styles.
- To integrate the above capabilities into a single web-based research workflow.

IV. Proposed System

Research Assistant is a web-based academic research support system that integrates academic information retrieval, Machine Learning, Natural Language Processing, PDF processing, and Generative AI. The system follows a client-server architecture consisting of a JavaScript-based frontend and a Python FastAPI backend. The backend coordinates local ML/NLP processing and communicates with external services such as the arXiv API and Groq API.

A. System Architecture

The frontend provides interfaces for semantic paper search, PDF analysis, summarization, literature-review generation, plagiarism checking, text humanization, conversational assistance, and citation generation. User inputs are transmitted to the FastAPI backend through REST-based HTTP requests.

The backend uses the arXiv API for academic-paper retrieval, PyMuPDF for PDF text extraction, the pretrained `all-MiniLM-L6-v2` Sentence Transformer for semantic processing, the Pegasus paraphrasing model for humanization, and the `openai/gpt-oss-120b` model through the Groq API for Generative AI tasks.

The overall processing flow is:

User → Frontend → FastAPI Backend → Processing Module → ML/NLP/AI Service → Result

B. Semantic Research-Paper Search

The semantic search module retrieves research papers from arXiv and ranks them according to their semantic relevance to the user's query. The title and abstract of each retrieved paper are combined and encoded using `all-MiniLM-L6-v2`. The user query is encoded using the same model.

Cosine similarity is calculated between the query embedding $\mathbf{q}$ and each paper embedding $\mathbf{p}$:

$$S(\mathbf{q}, \mathbf{p}) = \frac{\mathbf{q} \cdot \mathbf{p}}{\|\mathbf{q}\|\|\mathbf{p}\|} \tag{1}$$

The papers are sorted according to their similarity scores, and the highest-ranked results are returned to the frontend. This enables semantic retrieval beyond exact keyword matching.

C. PDF Document Analysis

Uploaded research papers are processed using PyMuPDF to extract textual content. The extracted content is subsequently supplied to the required analysis modules. This allows a research paper to be transformed from an uploaded document into machine-readable text that can be summarized or analyzed.

The system also handles large extracted documents and provides document-level information such as page count and extracted character count.

D. Summarization and Literature Review

The summarization module uses Generative AI to generate structured summaries from supplied research content. The literature-review module combines academic retrieval with Generative AI. Relevant papers are retrieved from arXiv and their metadata and abstracts are supplied to the language model. The generated review can contain structured sections covering research themes, trends, gaps, future directions, and conclusions.

E. Plagiarism-Oriented Textual Similarity

The plagiarism module combines lexical and semantic similarity. N-gram overlap is used to identify matching phrases, while the Sentence Transformer is used for sentence-level semantic comparison.

The combined score is calculated as:

$$S_{\text{combined}} = 0.6S_{\text{ngram}} + 0.4S_{\text{semantic}} \tag{2}$$

The system also retrieves potentially related papers from arXiv for comparison. To improve robustness, generated arXiv search queries are sanitized before being sent to the external API. HTTP responses and XML content are validated before parsing, and invalid or unavailable arXiv responses are handled gracefully rather than causing the plagiarism operation to fail.

F. Text Humanization

The Humanizer is designed as a meaning-preserving paraphrasing component rather than an AI-authorship detector. It supports three writing styles:

- Academic Style
- Casual Student Style
- Conversational Natural

The module uses the pretrained `tuner007/pegasus_paraphrase` sequence-to-sequence model to generate paraphrase candidates. The `all-MiniLM-L6-v2` Sentence Transformer is then used to evaluate semantic similarity between the original and generated sentences.

The Humanizer follows an $N \rightarrow N$ sentence invariant. If the input contains N sentences, each sentence is processed

independently and exactly one output sentence is generated or retained for each input position. Sentences are not intentionally merged, deleted, reordered, or summarized.

For each input sentence, multiple Pegasus candidates are generated. Candidates with semantic similarity below a threshold of 35% are rejected. The remaining candidates are evaluated using a weighted score consisting of semantic similarity, lexical diversity, and quality heuristics:

$$S_{\text{final}} = 0.70S_{\text{semantic}} + 0.15S_{\text{diversity}} + 0.15S_{\text{quality}} \quad (3)$$

The candidate with the highest valid score is selected. The original sentence is not included in the normal candidate competition.

G. Protected Information Validation

A major enhancement of the Humanizer is protected-information validation. Generative paraphrasing models may occasionally alter numerical values, technical identifiers, or domain-specific terminology. To reduce this risk, the system extracts important protected items from the original sentence before candidate selection.

Protected items may include:

- Numerical values and percentages.
- Model and package names.
- Technical identifiers.
- Hyphenated technical expressions.
- Important domain-specific terminology.

Each generated candidate is checked against these protected items. A candidate that changes or removes important protected information is rejected. If all candidates for a sentence fail the validation requirements, the original sentence is retained for that position.

This mechanism prioritizes factual and technical information preservation over forcing a paraphrase.

V. Methodology

The methodology combines pretrained Machine Learning models, traditional NLP techniques, information retrieval, and Generative AI. Each component is selected according to the type of research task being performed.

A. Semantic Embedding and Similarity

The primary Machine Learning component is the pretrained `all-MiniLM-L6-v2` Sentence Transformer. Research queries and paper title-abstract representations are converted into fixed-dimensional semantic embeddings. Cosine similarity is then used to measure their semantic relationship.

The same model is reused for sentence-level comparison during plagiarism analysis and for evaluating generated paraphrase candidates in the Humanizer.

The model is not trained or fine-tuned as part of the project. Instead, pretrained weights are integrated into the application.

B. N-Gram Similarity

The plagiarism module also applies lexical n-gram analysis. Submitted text and comparison material are divided into consecutive word sequences using 3-grams and 4-grams. The overlap between the two sets provides a measure of direct or near-direct phrase reuse.

If G_t and G_c represent the n-gram sets of the submitted and comparison texts, respectively, the overlap can be expressed as:

$$S_{\text{ngram}} = \frac{|G_t \cap G_c|}{|G_t|} \times 100 \quad (4)$$

This lexical score is combined with semantic similarity to produce the plagiarism-oriented score.

C. Generative AI Processing

Generative AI operations use the `openai/gpt-oss-120b` model through the Groq API. A common backend helper manages communication with the model, while individual modules provide task-specific prompts.

The general process is:

Input → Task Prompt → Groq API → GPT-OSS 120B → Generated Output

The model is used for tasks such as summarization, PDF analysis, literature-review generation, conversational research assistance, and interpretation of plagiarism-analysis results.

D. Pegasus Paraphrasing

The Humanizer uses the pretrained `tuner007/pegasus_paraphrase` model. The model receives individual sentences and generates alternative formulations.

Generation parameters are adjusted according to the selected writing style. Academic Style uses more conservative beam-search generation, while Casual Student Style and Conversational Natural use sampling-based generation to encourage greater variation.

The generated candidates are then evaluated using the Sentence Transformer model. This creates a separation between generation and semantic validation: Pegasus produces candidate text, while MiniLM evaluates whether the candidate remains sufficiently close in meaning to the original.

E. Humanizer Candidate Scoring

The Humanizer uses three main candidate-quality components.

Semantic similarity measures how closely the candidate preserves the meaning of the original sentence.

Lexical diversity rewards meaningful rewriting rather than simple copying.

Quality heuristics consider characteristics such as repetition and approximate length preservation.

The combined score is:

$$S_{\text{final}} = 0.70S_{\text{semantic}} + 0.15S_{\text{diversity}} + 0.15S_{\text{quality}} \quad (5)$$

Candidates with semantic similarity below 35% are rejected. Protected-information validation is applied independently to ensure that numerical and technical information is not corrupted.

F. Literature Review Generation

For literature-review generation, the user's topic is submitted to the arXiv search service and relevant papers are retrieved. Selected metadata and abstract information are combined and supplied to the Generative AI model. The generated response is structured into research-oriented sections such as introduction, thematic analysis, research trends, research gaps, future directions, conclusion, and references.

This retrieval-plus-generation workflow allows the system to use retrieved scholarly information as the basis for generated research assistance.

G. Citation Generation

Citation generation is implemented using deterministic Python formatting functions. Bibliographic information such as title, authors, year, venue, volume, and pages is formatted according to the selected citation style.

The system supports:

- APA
- IEEE
- MLA
- Chicago
- Harvard

VI. Implementation

Research Assistant is implemented as a web-based client-server application. The frontend is developed using HTML, CSS, and JavaScript, while the backend is implemented in Python using FastAPI. The major components are summarized in Table I.

TABLE I
TECHNOLOGY STACK

Component	Technology
Frontend	HTML, CSS, JavaScript
Backend	Python, FastAPI
Paper Retrieval	arXiv API
PDF Processing	PyMuPDF
Semantic Model	all-MiniLM-L6-v2
Paraphrasing Model	Pegasus Paraphrase
Generative AI	GPT-OSS 120B
AI API	Groq API
Data Validation	Pydantic

A. Backend and API Implementation

The FastAPI backend exposes REST endpoints for the major functions of the application. The frontend sends JSON requests for text-based operations and multipart form-data requests for PDF uploads. The backend validates input, performs the required processing, and returns structured JSON responses.

The major endpoints are summarized in Table II.

The previous AI Detection endpoint was intentionally removed from the final implementation and is no longer part of the application.

TABLE II
MAJOR BACKEND API ENDPOINTS

Endpoint	Function
`/semantic-search`	Semantic paper search
`/summarize`	Text summarization
`/upload-pdf`	PDF analysis and extraction
`/check-plagiarism`	Plagiarism-oriented analysis
`/humanize`	ML-based text humanization
`/chat`	Research assistant chat
`/literature-review`	Literature review generation
`/cite`	Citation generation

B. ML and Generative AI Integration

The Sentence Transformer model is loaded by the backend when semantic processing is required and reused for subsequent similarity operations. It is integrated into the semantic-search, plagiarism-analysis, and Humanizer pipelines.

The Pegasus paraphrasing model is loaded for Humanizer operations. Candidate generation is performed on individual sentences, followed by MiniLM-based semantic evaluation and protected-information validation.

Generative AI functionality is provided through the Groq API using the `openai/gpt-oss-120b` model. A common backend function handles communication with the model, while individual modules provide task-specific prompts.

Academic-paper retrieval is implemented through HTTP requests to the arXiv API, and returned paper metadata is parsed by the backend. Uploaded PDFs are processed using PyMuPDF before their extracted content is passed to the required analysis module.

C. Frontend Result Rendering

AI-generated responses can contain Markdown formatting such as bold text, italic text, headings, bullet lists, and numbered lists. A centralized frontend renderer was implemented to provide consistent formatting across PDF analysis, summarization, literature review, plagiarism analysis, and Ask AI results.

Before Markdown transformations are applied, AI output is HTML-escaped. This prevents raw HTML or script content from being interpreted as executable markup. The renderer then converts only the supported Markdown structures into controlled HTML elements.

This approach provides consistent presentation while reducing the risk of arbitrary HTML execution from generated content.

VII. Results and Discussion

The developed Research Assistant system was evaluated using representative research queries, academic papers, technical text, and functional test inputs. The evaluation focused on functional correctness, semantic processing, paraphrasing behavior, information preservation, response generation, and module-level integration.

A. Semantic Search Results

The semantic search module was tested using the query "machine learning in healthcare". The system successfully retrieved research papers from arXiv and ranked them according to semantic similarity.

In an observed test, the highest-ranked result achieved a similarity score of 70.2%, while another retrieved paper achieved 60.3%. These values represent system-generated semantic relevance indicators rather than standardized retrieval accuracy.

The results demonstrate that the Sentence Transformer-based retrieval mechanism can identify papers that are semantically related to a research query even when the wording is not identical.

B. PDF Analysis Results

The PDF-processing module was tested using a research paper named `P01.pdf`. The document contained 15 pages and approximately 80,933 extracted characters.

The extraction process completed successfully and the resulting content was passed to the analysis pipeline. The system generated structured research-paper information including title, authors, summary, and other analysis sections.

The result demonstrates the ability of the system to transform a research PDF into machine-readable text and subsequently apply multiple research-support operations.

C. Plagiarism Analysis Results

The plagiarism module was evaluated using small textual inputs, technical text, and inputs containing potentially problematic symbols.

A small-text test completed successfully with a reported similarity score of 7.8%. A technical-text test produced a score of 18.5%, while a test containing operators and special characters produced a score of 6.6%. These scores are system-generated similarity indicators and are dependent on the retrieved comparison material.

An important robustness issue was identified during development when a large PDF-derived query caused arXiv to return an HTML error response instead of XML. The application initially attempted to parse this response as XML, resulting in the error "syntax error: line 1, column 0". The final implementation sanitizes arXiv queries, validates HTTP responses, and catches XML parsing errors. Subsequent regression testing confirmed that the plagiarism endpoint returned successfully without this error.

D. Humanizer Results

The Humanizer was evaluated using three supported writing styles:

- Academic Style
- Casual Student Style
- Conversational Natural

The target test text concerned the use of Machine Learning in healthcare. All three styles produced paraphrased outputs while preserving the sentence structure.

A technical-information preservation test contained 15 protected items, including numerical values, percentages, model names, technical terms, and dimensional information. All 15 protected items were preserved across all three styles, resulting in 15/15 or 100% protected-information preservation for each style.

The observed semantic similarity for this technical test ranged from 99.9% to 100.0%. These values indicate semantic similarity between the tested original and generated text and should not be interpreted as general model accuracy.

A short-input test containing 41 characters was also successfully processed after reducing the minimum input threshold from 50 to 20 characters. The Academic and Casual Student styles produced a semantic similarity of approximately 95.8%, while the Conversational Natural style produced approximately 95.3%.

A long-text test containing 12 sentences confirmed the $N \rightarrow N$ sentence invariant for all three styles:

- Academic Style: 12 input sentences $\rightarrow$ 12 output sentences, 98.2% meaning preservation.
- Casual Student Style: 12 input sentences $\rightarrow$ 12 output sentences, 96.4% meaning preservation.
- Conversational Natural: 12 input sentences $\rightarrow$ 12 output sentences, 98.0% meaning preservation.

The three styles also generated distinct outputs, demonstrating that style-specific generation parameters influence the resulting paraphrases.

E. Literature Review and Generative AI Results

The literature-review module was tested using the topic "machine learning in healthcare". The system successfully retrieved relevant research papers and generated a structured literature review using the Generative AI pipeline.

The review included structured sections and citations corresponding to the retrieved papers. In observed testing, the literature-review endpoint completed successfully in approximately 7.9 seconds.

The summarization endpoint also completed successfully in approximately 2.6 seconds during regression testing.

F. Ask AI and Citation Results

The Ask AI module was tested using research-oriented questions. The endpoint successfully generated responses using the configured Generative AI model. An observed regression test completed in approximately 2.7 seconds.

The citation generator was tested with APA, IEEE, MLA, Chicago, and Harvard formats. All five formats generated valid citation outputs, with observed response times of approximately two seconds.

G. Full-System Functional Validation

A final QA pass was performed across the major application modules. A total of 31 tests were considered, of which 27 were executed successfully and 4 were blocked by environmental limitations related to browser automation or test-file path

resolution. No genuine application failures were identified in the executed tests.

The regression suite confirmed successful operation of:

- Semantic research-paper search.
- PDF analysis and summarization.
- Literature-review generation.
- Plagiarism analysis.
- Humanization.
- Ask AI.
- Citation generation.

The AI Detection module was also verified as removed from the final application, with the former `/detect-ai` endpoint returning HTTP 404.

TABLE III
OBSERVED REGRESSION TEST PERFORMANCE

Module	Time	Status
Semantic Search	4.3 s	PASS
Summarization	2.6 s	PASS
Literature Review	7.9 s	PASS
Plagiarism Checker	6.2 s	PASS
Humanizer	7.6 s	PASS
Ask AI	2.7 s	PASS
Citation Generator	2.0 s	PASS

The reported values demonstrate that the application can execute the major research-support operations within practical response times during the observed tests. These measurements are implementation-level observations and may vary with input size, external API response time, and system resources.

H. Overall Discussion

The experimental results demonstrate that the proposed system successfully integrates multiple research-support capabilities into a single workflow. The semantic-search module provides embedding-based paper ranking, the PDF module converts research documents into analyzable text, the plagiarism module combines lexical and semantic similarity, and the Humanizer combines Transformer-based paraphrasing with semantic validation.

The Generative AI layer provides summarization, literature-review generation, conversational research assistance, and analytical interpretation. The centralized Markdown renderer improves the presentation of generated responses while applying HTML escaping before controlled formatting.

The evaluation is primarily a functional and system-level evaluation rather than a benchmark-based ML evaluation using a large labeled dataset. Therefore, numerical values such as semantic similarity, plagiarism similarity, and paraphrasing quality should be interpreted as system-generated indicators rather than standardized accuracy measurements.

VIII. LIMITATIONS AND FUTURE SCOPE

Although Research Assistant integrates several academic research functions into a single platform, the current implementation has several limitations.

The semantic-search module primarily depends on papers available through the arXiv API and therefore does not represent the complete academic literature available through other scholarly databases. Similarly, the plagiarism module compares submitted content with retrieved research material and should not be considered equivalent to large commercial plagiarism-detection databases.

The Generative AI components depend on an external API and therefore may be affected by model availability, network conditions, service limits, and changes in external model catalogs. The project experienced a model-access issue during development, which was resolved by migrating from the unavailable `llama-3.3-70b-versatile` configuration to the accessible `openai/gpt-oss-120b` model.

The Humanizer uses a pretrained paraphrasing checkpoint and does not perform domain-specific fine-tuning. Although protected-information validation significantly improves preservation of numerical and technical information in the tested cases, paraphrasing quality may still vary across domains, writing styles, and uncommon terminology.

PDF analysis depends on successful text extraction. Scanned or image-based documents may require OCR, which is not part of the current implementation.

The current evaluation is primarily functional and does not constitute a large-scale benchmark against manually labeled academic datasets. More extensive evaluation would be required to establish standardized retrieval, paraphrasing, plagiarism-detection, or summarization performance.

Future development can extend the system by integrating additional academic databases and a dedicated vector database for large-scale semantic retrieval. Retrieval-Augmented Generation (RAG) could be incorporated to provide more context-grounded responses from uploaded research documents [3]. Domain-specific fine-tuning of embedding or paraphrasing models could improve performance for specialized research fields. OCR support could improve processing of scanned documents. Additional features could include citation-graph analysis, research-gap visualization, document comparison, multilingual support, and benchmark-based evaluation using domain-specific datasets.

IX. CONCLUSION

This paper presented Research Assistant, a web-based academic research support system integrating Machine Learning, Natural Language Processing, information retrieval, PDF processing, and Generative AI. The system provides an integrated workflow for semantic research-paper search, PDF analysis, summarization, literature-review generation, plagiarism-oriented similarity analysis, text humanization, conversational research assistance, and citation generation.

The primary Machine Learning component uses the pretrained `all-MiniLM-L6-v2` Sentence Transformer to generate semantic embeddings for research retrieval, similarity analysis, and paraphrase evaluation. Traditional NLP techniques such as n-gram matching complement the semantic approach. The Humanizer uses the pretrained `tuner007/pegasus_paraphrase` model for sentence-level paraphrasing and applies semantic scoring and protected-

information validation to improve meaning and technical-information preservation. The Generative AI layer uses the `openai/gpt-oss-120b` model through the Groq API for research-oriented generation tasks.

Functional validation demonstrated that the major application modules operated successfully during the executed regression tests. The Humanizer preserved all tested protected technical information, maintained the $N \rightarrow N$ sentence invariant on long input, and produced distinct outputs for the three supported writing styles. The plagiarism module was made robust against malformed arXiv responses, while centralized Markdown rendering improved the presentation of AI-generated results.

The project demonstrates the potential of combining semantic Machine Learning models, traditional NLP techniques, academic information retrieval, Transformer-based paraphrasing, and Generative AI within a unified academic research workflow. Although further benchmark-based evaluation and domain-specific improvements are required for broader validation, the implemented system provides a practical foundation for integrated academic research assistance.

REFERENCES

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[3] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[4] arXiv, "arXiv API User's Manual," Cornell University.

[5] Artifex Software, "PyMuPDF Documentation."

[6] J. Zhang et al., "PEGASUS: Pre-training with Extracted Gap-sentences for Abstractive Summarization," in *Proceedings of the 37th International Conference on Machine Learning*, 2020.